Chapter 7

LP PREPROCESSING

Large scale LP problems are generally not solved in the form they are submitted to an optimization system. There are a number of things that can be done to the original problem to increase the chances of solving it reliably and efficiently. Problems usually can be simplified and their numerical characteristics improved. The activities dealing with these issues are generally called *preprocessing*. The two main actions are *presolve* which attempts simplification and *scaling* which tries to improve the numerical characteristics of the problem. During presolve the problem undergoes equivalent transformations. In Chapter 1, when the LP problem was brought to a computationally more suitable form, we did something that also can be viewed as preprocessing. However, because of its nature it had to be discussed separately.

When a solution is obtained the values of the variables of the original problem have to be restored. This procedure is called *postsolve*. Though it is not a real preprocessing activity this chapter is the best place to discuss it. Therefore, we go through each of the above outlined procedures in some detail that is sufficient for a successful implementation of the most important elements of them. Additionally, we provide references to original research papers for readers interested in more details.

7.1. Presolve

Large scale LP problems are usually generated by some general purpose modeling systems or purpose built programs. As the problem instances are much larger than humans can comprehend it is inevitable that some weaknesses in a model remain overlooked. Examples are the inclusion of empty rows/columns, redundant constraints that unnecessarily increase the size and complexity of the problem. If they are de-

tected and removed a smaller problem can be passed over to the solver. Several other beneficial effects can be achieved by suitably designed and implemented presolve algorithms.

In general, the main objectives of presolve are to

1 reduce the size of the problem,

2 improve the numerical characteristics, and

3 detect infeasibility or unboundedness without solving the problem.

Studying the output of presolve can be instructive. It provides a useful feedback to the modeler and can contribute to improved problem solving already in the modeling phase.

As in many other cases, there is a trade-off between the amount of work and the effectiveness of presolve. The techniques applied consist of a series of simple tests. We can expect more problem reduction if more tests are performed. However, there is no performance guarantee.

In their pioneering work, Brearley, Mitra and Williams [Brearley et al., 1975] suggested the first tests that are capable of (i) detecting and removing redundant constraints, (ii) replacing constraints by simple bounds, (iii) replacing columns by bounds on shadow prices, (iv) fixing variables at their bounds, and (v) removing or tightening redundant bounds. They also noticed the different requirements of the LP and Mixed Integer LP (MILP) presolves.

The importance of the ideas of their work has grown substantially as its benefits have become increasingly evident. Nowadays some version of a presolve is a 'must' for an advanced implementation of the simplex method. It is very likely based on [Brearley et al., 1975].

Other important contributions to the ideas of presolve are due to Tomlin and Welch [Tomlin and Welch, 1983a, Tomlin and Welch, 1983b, Tomlin and Welch, 1986], Fourer and Gay [Fourer and Gay, 1993], Andersen and Andersen [Andersen and Andersen, 1995], and Gondzio [Gondzio, 1997]. The latter two also elaborate on the special requirements posed by the interior point methods for linear programming. There is an interesting idea of aggregation that reduces the size of the problem by aggregating constraints. Liesegang gives an in-depth discussion of this topic in [Liesegang, 1980].

To introduce the tests of presolve we use the LP problem with general lower bounds:

$$\begin{aligned} \min \quad & \mathbf{c}^T\mathbf{x} + c_0 && (7.1)\\ \text{s. t.} \quad & \mathbf{A}\mathbf{x} = \mathbf{b} && (7.2)\\ & \boldsymbol{\ell} \leq \mathbf{x} \leq \mathbf{u}, && (7.3) \end{aligned}$$

allowing some ℓ_j and u_j to be $-\infty$ and $+\infty$, respectively. The c_0 term in the objective function represents its initial value. In general, it is assumed zero and is omitted. However, in the context of presolve it is worth including it explicitly.

The dual of (7.1) – (7.3) is

$$\begin{aligned} \max \quad & \mathbf{b}^T\mathbf{y} + \boldsymbol{\ell}^T\mathbf{d} - \mathbf{u}^T\mathbf{w} && (7.4)\\ \text{s. t.} \quad & \mathbf{A}^T\mathbf{y} + \mathbf{d} - \mathbf{w} = \mathbf{c} && (7.5)\\ & \mathbf{d}, \mathbf{w} \geq \mathbf{0}. && (7.6) \end{aligned}$$

This form can be obtained by using the developments in section 10.1. All the vectors in (7.1) through (7.6) are of compatible dimension with $\mathbf{A} \in \mathbb{R}^{m \times n}$. It is to be noted that the dual variables, y_i, $i = 1, \ldots, m$, are unrestricted in sign.

During presolve some row/column eliminations are identified. They are performed with immediate effect. It means that these entities (rows or columns) and all their nonzero elements are left out from further investigations as if they were not there. Therefore, the running indices in the summations are understood to cover their currently valid range. Also, some updating of the right-hand side or the objective function may be necessary. They are also performed and the b_i or c_j coefficients always refer to their current value.

Several tests are based on the smallest and largest values a constraint can take. To determine them, we define the index sets of the positive and negative coefficients in each row of $\mathbf{A}$ in (7.2).

$$\mathcal{P}_i = \{j : a_j^i > 0\}, \quad (7.7)$$

$$\mathcal{N}_i = \{j : a_j^i < 0\}, \quad (7.8)$$

for the available rows. In this way the smallest value, which will be referred to as *lower limit*, is

$$\underline{b}_i = \sum_{j \in \mathcal{P}_i} a_j^i \ell_j + \sum_{j \in \mathcal{N}_i} a_j^i u_j, \quad (7.9)$$

and the largest value, *upper limit*, is

$$\overline{b}_i = \sum_{j \in \mathcal{P}_i} a_j^i u_j + \sum_{j \in \mathcal{N}_i} a_j^i \ell_j. \quad (7.10)$$

If infinities appear among the ℓ_j and u_j bounds with nonnegative a_j^i the sums they contribute to will be infinite. In such a case it is worth keeping a count of the number of contributing infinities.

7.1.1 Contradicting individual bounds

This a perhaps the simplest possible test. If $\ell_j > u_j$ for any j the problem is obviously infeasible. It is worth testing all $j = 1, \ldots, n$ to detect all contradictions of this type.

7.1.2 Empty rows

If the only nonzero in a row i is the '1' of the implied logical variable z_i in (7.2) then all the structural coefficients are zero. Such a row is called an *empty row.* It may be included in the model inadvertently or can be the result of other reductions during presolve. Now we have $z_i = b_i$. Depending on the type of z_i (expressing the type of the constraint) the b_i may be a feasible value for z_i, in which case constraint is redundant and the actual row size of the problem is reduced by one. If b_i is an infeasible value for z_i the constraint is a contradiction and the problem is infeasible.

7.1.3 Empty columns

If column j is empty the corresponding variable does not consume any resources, i.e., does not influence the joint constraints. Depending on the value of c_j the following cases can occur.

1 $c_j = 0$ which implies x_j can take any feasible value. Therefore, we can fix it within $\ell_j \leq u_j$ and take it out of the problem. This fixing does not cause any change in the objective function.

2 $c_j > 0$ which entails x_j can be fixed at its lower bound ℓ_j as this results in the best contribution to the objective function (minimization). If $\ell_j = -\infty$ the solution is unbounded.

3 $c_j < 0$ which implies x_j should be set to its upper bound u_j to contribute the best to the objective. If $u_j = +\infty$ the solution is unbounded.

Fixing a variable with $c_j \neq 0$ causes a change in the objective function. Formally, if we fix $x_j = t$, where t is the chosen value, then in the objective we have $c_0 := c_0 + c_j t$.

7.1.4 Singleton rows

If only one structural coefficient a_j^i is nonzero in row i then it is called a *singleton row.* Such rows can be removed from the problem. If the constraint is an equality the value of x_j is uniquely determined by $x_j = b_i/a_j^i$. If $x_j \notin [\ell_j, u_j]$ then the problem is infeasible. Otherwise, x_j is fixed at this value and is removed from the problem.

Whenever row i is a type-3 (non-binding) constraint x_j can be fixed at any feasible value and removed.

If row i is an inequality constraint then it defines a lower or upper bound for x_j. If a newly evaluated bound is tighter than the currently known one it can be used to replace the old one.

Assuming $a_j^i x_j \leq b_i$ we have the following possibilities.

1 If $a_j^i > 0$ then $x_j \leq \bar{u}_j = b_i/a_j^i$.

2 If $a_j^i < 0$ then $x_j \geq \bar{\ell}_j = b_i/a_j^i$.

In the opposite case, when $a_j^i x_j \geq b_i$,

1 If $a_j^i > 0$ then $x_j \geq \bar{\ell}_j = b_i/a_j^i$.

2 If $a_j^i < 0$ then $x_j \leq \bar{u}_j = b_i/a_j^i$.

If a lower and an upper limit are available for row i such that $\underline{b}_i \leq a_j^i x_j \leq \bar{b}_i$ and any or both of $\underline{b}_i$ and $\bar{b}_i$ are finite then the above relations can be used to evaluate individual bounds on x_j. The new bounds are defined to be the better of the old and the new ones:

$$\ell_j := \max\{\ell_j, \bar{\ell}_j\}, \tag{7.11}$$

$$u_j := \min\{u_j, \bar{u}_j\}. \tag{7.12}$$

7.1.5 Removing fixed variables

A variable becomes fixed if at some stage (or at the beginning) its lower and upper bounds are equal, $\ell_j = x_j = u_j$. Another reason for fixing variables was shown in section 7.1.3. A fixed variable can be removed from the problem but the constant term of the objective and the limits of the joint constraints have to be updated in a way similar to the lower bound translation in (1.13).

To be more specific, let x_j be fixed at a value t, i.e., $x_j = t$. The contribution of x_j to the objective function is $c_j t$ which can be accounted for in the constant term as $c_0 := c_0 + c_j t$. The constant vector $t\mathbf{a}_j$ is taken out of $\mathbf{Ax}$ and is accounted for in the limits of the joint constraints as

$$\overline{\mathbf{b}} := \overline{\mathbf{b}} - t\mathbf{a}_j, \tag{7.13}$$

$$\underline{\mathbf{b}} := \underline{\mathbf{b}} - t\mathbf{a}_j. \tag{7.14}$$

In this way the number of variables is reduced by one with each fixing.

7.1.6 Redundant and forcing constraints

Any $\mathbf{x}$ satisfying (7.2) and (7.3) also satisfies

$$\underline{b}_i \leq \sum_j a_j^i x_j \leq \bar{b}_i, \quad \forall i. \tag{7.15}$$

Assume constraint i was originally a '$\leq$' type constraint:

$$\sum_j a_j^i x_j \leq b_i. \tag{7.16}$$

Comparing b_i with the limits $\underline{b}_i$ and $\bar{b}_i$ enables us to draw some conclusions.

1 $b_i < \underline{b}_i$: Constraint (7.16) cannot be satisfied because even the smallest value of $\sum_j a_j^i x_j$ is greater than the upper bound b_i. Thus the problem is infeasible.

2 $b_i = \underline{b}_i$: We say that constraint (7.16) is a *forcing* one as it forces all variables involved in the definition of $\underline{b}_i$ to be fixed at those values assumed in the definition. Namely, $x_j = \ell_j$, $j \in \mathcal{P}_i$ and $x_j = u_j$, $j \in \mathcal{N}_i$. All the fixed variables and also constraint i are then removed from the problem. This action reduces the number of constraints by one and the number of variables by $|\mathcal{P}_i| + |\mathcal{N}_i|$.

3 $b_i \geq \bar{b}_i$: Constraint i cannot exceed its computed upper limit $\bar{b}_i$, therefore, it is redundant and can be removed. As a result, the number of constraints is reduced by one.

4 $\underline{b}_i < b_i < \bar{b}_i$: Constraint i cannot be eliminated.

If the original constraint is of type '$\geq$' it can be multiplied by -1 and the above procedure can be applied. The other possibility is to evaluate the different cases directly from $\sum_j a_j^i x_j \geq b_i$ using arguments similar to the above ones.

7.1.7 Tightening individual bounds

The computed limits on a constraint can often be used to derive individual bounds on the participating variables. This is quite straightforward if one or both limits are finite. Gondzio [Gondzio, 1997] noticed that it is possible to determine finite bounds if there is only one infinite bound in the equation defining $\underline{b}_i$ or $\bar{b}_i$. First we discuss the finite case followed by Gondzio's the extension.

For simplicity, we restrict ourselves to the '$\leq$' type constraints (a representative of them is (7.16)) and show the results in this context.

Assume $\underline{b}_i$ is finite and in its definition we have variable x_k such that $k \in \mathcal{P}_i$ which is at its lower bound, $x_k = \ell_k$. If we allow any single variable in $\mathcal{P}_i$ to move away from ℓ_k we obtain

$$\underline{b}_i + a_k^i(x_k - \ell_k) \leq \sum_j a_j^i x_j \leq b_i, \quad k \in \mathcal{P}_i, \tag{7.17}$$

and, in a similar fashion, if x_k, $k \in \mathcal{N}_i$, is at its upper bound,

$$\underline{b}_i + a_k^i(x_k - u_k) \leq \sum_j a_j^i x_j \leq b_i, \quad k \in \mathcal{N}_i. \tag{7.18}$$

From equation (7.17) a lower and from (7.18) an upper bound can be obtained for x_k

$$x_k \leq \bar{u}_k = \ell_k + \frac{b_i - \underline{b}_i}{a_k^i}, \quad k \in \mathcal{P}_i, \tag{7.19}$$

and

$$x_k \geq \bar{\ell}_k = u_k + \frac{b_i - \underline{b}_i}{a_k^i}, \quad k \in \mathcal{N}_i. \tag{7.20}$$

If any of the computed bounds $\bar{\ell}_k$ and $\bar{u}_k$ is tighter than the currently used lower or upper bound the new replaces the old one in the same way as in (7.11) and (7.12). We may also obtain that $\bar{\ell}_k > \bar{u}_k$, in which case the problem is infeasible.

The above calculations are performed for each row which has a zero infinity count. Gondzio proposed an extension of the above for rows that have infinity count equal one.

It is assumed that $\underline{b}_i$ is not finite and there is only one infinite bound, say $\ell_k = -\infty$ for some $k \in \mathcal{P}_i$ or $u_k = +\infty$ for some $k \in \mathcal{N}_i$, in its definition, see (7.9). Now, equations (7.17) and (7.18) can be replaced by

$$a_k^i x_k + \sum_{j \in \mathcal{P}_i \setminus \{k\}} a_j^i \ell_j + \sum_{j \in \mathcal{N}_i} a_j^i u_j \leq \sum_j a_j^i x_j \leq b_i, \quad \text{if } k \in \mathcal{P}_i, \tag{7.21}$$

or

$$a_k^i x_k + \sum_{j \in \mathcal{P}_i} a_j^i \ell_j + \sum_{j \in \mathcal{N}_i \setminus \{k\}} a_j^i u_j \leq \sum_j a_j^i x_j \leq b_i, \quad \text{if } k \in \mathcal{N}_i. \tag{7.22}$$

From (7.21) an upper bound can be derived for x_k

$$x_k = \bar{u}_k \leq \frac{1}{a_k^i} \left(b_i - \sum_{j \in \mathcal{P}_i \setminus \{k\}} a_j^i \ell_j - \sum_{j \in \mathcal{N}_i} a_j^i u_j \right) \tag{7.23}$$

and from (7.22) a lower bound can be obtained for x_k

$$x_k = \bar{\ell}_k \geq \frac{1}{a_k^i}\left(b_i - \sum_{j\in\mathcal{P}_i} a_j^i \ell_j - \sum_{j\in\mathcal{N}_i\setminus\{k\}} a_j^i u_j\right) \tag{7.24}$$

Again, if any of the computed bounds $\bar{\ell}_k$ and $\bar{u}_k$ is tighter than the currently used lower or upper bound the new replaces the old one in the same way as in (7.11) and (7.12). We may also obtain that $\bar{\ell}_k > \bar{u}_k$, in which case the problem is infeasible.

Finally, we refer to the same two possibilities of handling the '$\geq$' type constraints as in the discussion of the forcing constraints.

It appears that the above extension is particularly useful if the problem contains free variables. Namely, in this case equations (7.17) and (7.18) do not give finite limits while (7.23) and (7.24) can. As a result, individual bounds can be generated for free variables that can lead even to their elimination. If the latter cannot be achieved then it is better to leave them as they were since free variables are very advantageous for the simplex method.

7.1.8 Implied free variables

For a moment, let ℓ_j and u_j denote the original (finite or infinite) individual bounds of x_j, and $\bar{\ell}_j$ and $\bar{u}_j$ the tightest computed ones. We call x_j an *implied free variable* if $[\bar{\ell}_j, \bar{u}_j] \subseteq [\ell_j, u_j]$ holds. In other words, if the row constraints guarantee that x_j stays within tighter limits than its original individual bounds then it can be treated as a free variable which is obviously beneficial.

In the special case when the implied free variable x_j is a column singleton with its nonzero entry in row i ($a_j^i \neq 0$) x_j can be removed from the problem. It is because its removal does not affect any other constraint. At the same time, constraint i can also be removed as it becomes a free row. The actions taken must be recorded because the removed row contains information about the relationship of the removed variable and the other variables that remain in the problem which must be taken into account during postsolve to determine the value of x_j in an optimal solution.

When a row is removed as a result of an implied column singleton we have to ensure that this change is accounted for also in the objective function. It can be seen from the dual constraint (7.5) that a singleton implied free variable x_j uniquely determines the i-th dual variable

$$y_i = \frac{c_j}{a_j^i}. \tag{7.25}$$

This value is always feasible because y_i is a free variable, see (7.5). To take the effect of this fixing into account the objective coefficients have to be modified

$$\bar{\mathbf{c}}^T = \mathbf{c}^T - y_i \mathbf{a}^i, \tag{7.26}$$

where $\mathbf{a}^i$ denotes row i of $\mathbf{A}$, as usual.

7.1.9 Singleton columns

The case of a free singleton column has been discussed in section 7.1.8. If the variable corresponding to a singleton column has at least one infinite bound we can draw conclusions using the optimality conditions as they impose sign constraints on the components of $\mathbf{d}$ and $\mathbf{w}$. Namely,

$$\begin{aligned} d_j &= 0 \quad \text{if} \quad \ell_j = -\infty, \\ w_j &= 0 \quad \text{if} \quad u_j = +\infty. \end{aligned}$$

It is easy to see that if exactly one of the bounds is finite the corresponding dual constraint in (7.5) becomes an inequality. If $u_j < +\infty$ then $w_j \geq 0$, therefore, by dropping w_j, we obtain

$$\mathbf{a}_j^T \mathbf{y} \leq c_j, \tag{7.27}$$

or if $\ell_j > -\infty$ then $d_j \geq 0$, thus

$$\mathbf{a}_j^T \mathbf{y} \geq c_j. \tag{7.28}$$

If $\mathbf{a}_j$ is a column singleton with a_j^i being the nonzero entry (7.27) and (7.28) are reduced to $a_j^i y_i \leq c_j$ or $a_j^i y_i \geq c_j$, respectively. In this way we can derive a bound for the y_i dual variable. Let $\gamma = c_j / a_j^i$. The following table shows the bounds generated by this column singleton

	$a_j^i > 0$	$a_j^i < 0$
$a_j^i y_i \leq c_j$	$y_i \leq \gamma$	$y_i \geq \gamma$
$a_j^i y_i \leq c_j$	$y_i \geq \gamma$	$y_i \leq \gamma$

7.1.10 Dominated variables

Using dual information there is a chance to draw further useful conclusions on the variables and constraints of the LP problem.

Let the bounds on the dual variables y_i be denoted by p_i and q_i such that

$$p_i \leq y_i \leq q_i, \quad i = 1, \ldots, m. \tag{7.29}$$

The index sets of the positive and negative entries in a column are defined as

$$\mathcal{P}'_j = \{i : a^i_j > 0\}, \tag{7.30}$$

$$\mathcal{N}'_j = \{i : a^i_j < 0\}, \tag{7.31}$$

for $j = 1, \ldots, n$. In this way the smallest value a dual constraint can take, which will be referred to as lower limit, is

$$\underline{c}_j = \sum_{i \in \mathcal{P}'_j} a^i_j p_i + \sum_{i \in \mathcal{N}'_j} a^i_j q_i, \tag{7.32}$$

and the largest value, upper limit, is

$$\overline{c}_j = \sum_{i \in \mathcal{P}'_j} a^i_j q_i + \sum_{i \in \mathcal{N}'_j} a^i_j p_i. \tag{7.33}$$

If infinities appear among the p_i and q_i bounds with nonnegative a^i_j the sums they contribute to will be infinite. It is worth keeping a count of the number of contributing infinities.

Any dual feasible solution $\mathbf{y}$ satisfies

$$\underline{c}_j \leq \sum_i a^i_j y_i \leq \overline{c}_j, \quad \forall j. \tag{7.34}$$

For simplicity, we assume $u_j = +\infty$, which means the corresponding dual constraint is $\mathbf{a}^T_j \mathbf{y} \leq c_j$ as shown in (7.28). Analyzing (7.34), the following cases can be distinguished and conclusions drawn.

1 $c_j < \underline{c}_j$: Dual constraint (7.28) cannot be satisfied, therefore the problem is dual infeasible.

2 $c_j > \overline{c}_j$: In this case d_j must be strictly positive to satisfy the j-th component of (7.5). It means x_j is *dominated*, it can be fixed at its finite lower bound, $x_j = \ell_j$ and eliminated from the problem. Obviously, if $\ell_j = -\infty$ the problem is dual unbounded.

3 $c_j = \overline{c}_j$: Dual constraint (7.28) is always satisfied with $d_j \geq 0$. The corresponding variable x_j is said to be *weakly dominated*. Under some further conditions (see [Gondzio, 1997]) a weakly dominated variable can also be fixed at its lower bound.

4 $\underline{c}_j \leq c_j < \overline{c}_j$: Variable x_j cannot be eliminated from the problem.

A similar analysis can be performed if we assume $\ell_j = -\infty$ in which case the relevant dual constraint, in inequality form, is (7.27).

7.1.11 Reducing the sparsity of *A*

As the advantages of sparsity of the constraint matrix **A** are overwhelming it is a good idea to try to increase it (i.e., decrease density). If it cannot be done logically anymore with the tests described so far we may turn to numerical elimination. Gondzio in [Gondzio, 1997] outlined an idea which is simple enough to implement but can also be effective in reducing the number of nonzeros. It is a heuristic algorithm with some minimum performance guarantee. It is based on the analysis of the sparsity pattern of **A**.

Any equality type constraint can be used to add a scalar multiple of it to any other constraint. This row will be referred to as pivot row. If row i is chosen for this purpose we select another row, say k, whose sparsity pattern is a superset of that of row i. In this case, the sparsity pattern of

$$\bar{\mathbf{a}}^k = \mathbf{a}^k + \gamma \mathbf{a}^i \tag{7.35}$$

is the same as $\mathbf{a}^k$. If $\bar{\mathbf{a}}^k$ replaces $\mathbf{a}^k$ in the LP problem the corresponding RHS element also changes: $\bar{b}_k = b_k + \gamma b_i$. Obviously, the problem with the new row k is equivalent with the original one.

By a suitable choice of γ in (7.35) we can achieve that at least one nonzero in $\mathbf{a}^k$ is eliminated. With some luck we can experience the elimination of some other nonzero elements. However, the guarantee is just one element per superset row. The beauty of this procedure is that it never produces fill-in and does not require additional storage.

While the idea is simple its efficient implementation is less trivial. The main difficulty lies in the search for rows with superset sparsity to a chosen row i. The choice of the pivot row can be critical. To enhance the chances of finding supersets it is advisable to start with rows having a small number of nonzeros.

The computational effort of identifying the supersets to row i can be reduced in the following way. Take the shortest column that intersects row i at a nonzero. Denote its index by j. Examine only those rows that have a nonzero in column j. If a row is not a superset of row i it usually becomes evident after checking only the first two or three elements. A row can also be rejected if it is shorter than row i. Figure 7.1 shows some typical situations that can occur.

For the efficient implementation of this procedure, a linked list of equality type rows is set up. They are the candidate pivot rows. The list is ordered by increasing number of nonzeros in the the rows. First, the shortest row is selected and the procedure is performed. Each processed pivot row is removed from the list and any equality type row in which a cancellation occurred reenters the list. When no superset can be found to

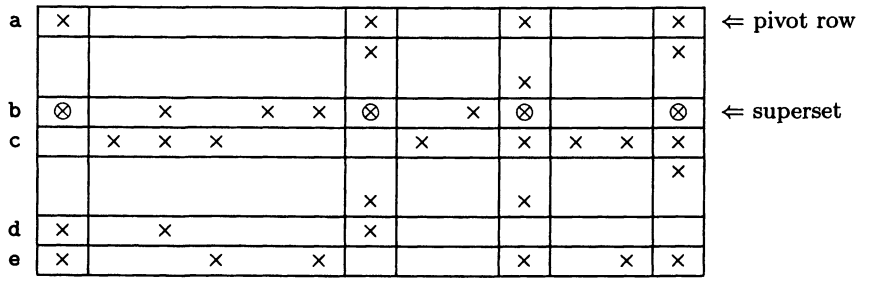

Figure 7.1. Identifying supersets to row **a** (pivot row). Column 1 is the shortest intersecting column and it is used to find supersets. Row **b** qualifies, row **c** is not considered because of the zero in column 1, row **d** fails as it is shorter than row **a**, and row **e** does not qualify as it fails to match the second nonzero of row **a**. One of positions ⊗ in row **b** can be eliminated.

any of the candidates (i.e., when the list becomes empty) the procedure stops.

7.1.12 The algorithm

The tests described in the previous sections can be combined to make a presolve algorithm. It is not necessary to include all discussed tests. On the other hand some more can be added from the references given in section 7.1. The more tests are included the more reduction can be expected. However, in this case computational effort and time will also increase.

After a setup when counts are initialized and sets are determined, the matrix is scanned several times. In every pass empty rows and columns are identified first and the appropriate actions taken. Then, each column is checked whether the corresponding variable is redundant or unbounded or provides a bound on the dual variable. Redundant variables are fixed at the determined level and removed from the problem. The remaining problem is modified to account for the changes. During a pass, individual bounds are calculated. If they are tighter they replace the old ones. At the end of the pass the new bounds are used to recompute the row limits and identify redundant or infeasible rows. Singleton rows are removed and the corresponding variables are fixed or individual bounds are generated. Also, implied free variables are identified.

Because of the iterative nature of the presolve algorithm a change in one pass may lead to further changes in the next. In general, if no reduction is achieved in two successive passes the algorithm terminates. However, theoretically it is possible that an infinite sequence of small

improvements in the bounds are generated, as pointed out by Fourer and Gay [Fourer and Gay, 1993] on a contrived example, in which case the algorithm cannot stop. Therefore, it is worth setting an upper limit on the number of passes.

7.1.13 Implementation

There are two main issues that require special attention for an efficient and effective presolve algorithm: the use of appropriate data structures and the proper handling of numerical operations.

Regarding data structures, certain techniques were already recommended in section 7.1.11. Some additional points are as follows. It is beneficial to keep **A** in both column- and rowwise forms. This double storage scheme can also be used during the simplex algorithm to enhance efficiency. Details of it are discussed in section 9.4.1. To avoid too much search linked lists can be set up that can be used in several tests and also in the housekeeping of the entire presolve. An important question is how the elimination of a row is performed. A good practice in the columnwise representation is the use of pointers to the start of the columns and keeping the lengths of the columns in terms of the number of nonzeros. In this case, if an element is to be deleted, it can be swapped with the last nonzero in the column and the length decreased by one. This way the ordering of the elements in the column changes but it is not a problem as no ordering is assumed by any of the algorithmic units of the simplex method.

It may come as a surprise but presolve is susceptible to numerical problems. The original matrix elements are stored in double precision and they remain unchanged during presolve. Their accuracy is determined during input conversion. The matrix elements may change if the algorithm for making **A** sparser is activated. The new bounds and limits are always obtained by calculations and may carry round-off and cancellation errors. Therefore, in evaluating, for instance, the forcing constraints, infeasibility or unboundedness of the problem, special care must be exercised. Fourer and Gay [Fourer and Gay, 1993] noticed that the use of directed, rounding when available with a processor, can increase the robustness of the procedure. When this facility is not available a tolerance should be used in comparisons of computed quantities. It may also help if the positive and negative terms of dot products are accumulated separately and added at the end. This can decrease the chances of cancellation error. Even more sophisticated techniques can be used if numerical problems persist.

Unfortunately, it is not easy to recognize that numerical errors caused problems (wrong conclusions) in presolve. In particular, wrong indica-

tion of infeasibility or unboundedness are not easy to verify without actually solving the un-presolved problem. Therefore, it is important that presolve is coded for robust operation. In this respect the proper use of tolerances is inevitable. They have to be used in comparisons for equality and also for inequality of computed quantities, like instead of $a = b$ we check whether $|a - b| < \epsilon$, where $\epsilon > 0$ is a tolerance in the magnitude of $10^{-12} - 10^{-10}$.

7.2. Scaling

Scaling in the LP context is a linear transformation of the matrix of constraints whereby each row and each column is multiplied by a real number called *scale factor*. It entails the adjustment of the other components of the LP problem, namely, right-hand side, objective function and bounds. Formally, scaling is equivalent to pre- and postmultiplying **A** by diagonal matrices, such that

$$\mathbf{RAC}\bar{\mathbf{x}} = \mathbf{Rb}, \tag{7.36}$$

where $\mathbf{R} = \text{diag}\langle R_1, \ldots, R_m \rangle$, $\mathbf{C} = \text{diag}\langle C_1, \ldots, C_n \rangle$, and $\mathbf{C}\bar{\mathbf{x}} = \mathbf{x}$. From the latter

$$\bar{\mathbf{x}} = \mathbf{C}^{-1}\mathbf{x}. \tag{7.37}$$

Obviously, the finite individual bounds of $\bar{x}_j$ are also transformed to u_j/C_j and, similarly, the explicit lower bounds, if given, to ℓ_j/C_j. The objective row is not scaled rowwise (implicitly, the factor is one). However the computed column scale factors can be applied to it. Thus, the objective coefficient of $\bar{x}_j$ becomes $c_j C_j$ (a bit of a notational hiccup). It ensures that

$$\bar{c}_j \bar{x}_j = c_j C_j x_j \frac{1}{C_j} = c_j x_j,$$

i.e., the objective values of the scaled and unscaled problems are the same. It can be useful when the iterations are monitored. The unit column vectors of logical variables are not scaled. Therefore, they are added to the LP problem after scaling.

The purpose of scaling is to improve the numerical characteristics of an LP problem and thus enable a safe solution. The importance of scaling in computational linear algebra has long been known. Unfortunately, its role in linear programming is not that well understood. There are examples where scaling makes it possible to solve an otherwise unsolvable problem but there are other examples where just the opposite occurs: a problem cannot be solved when scaled but becomes easily solvable without scaling. The main difference between traditional linear algebra and LP is that the former works with the matrix of a fixed set of equations

while in LP the set of (basic) equations keeps changing and each of them may require a different scaling. However, what can be done cheaply is a one-off scaling of **A** which then applies to all bases selected by the simplex method.

Good modeling practice can contribute to the good numerical characteristics of a problem. Namely, the proper choice of units of measurements of the variables usually leads to a balanced magnitude of the matrix elements which is believed, and in many cases proved, to be beneficial. Tomlin [Tomlin, 1975] has conducted a systematic review of scaling and attempted to make some recommendations. Though his results are inconclusive there are some interesting observations in his paper. We will use some of his arguments in the sequel.

It is quite difficult to characterize well scaled matrices. The opposite is somewhat easier. We say that a matrix is *badly scaled* if the magnitudes of the nonzero elements are vastly different. Practice has shown that such matrices are quite difficult to process (solve systems of equations with them, factorize, invert). As a contraposition, we can say (or believe) that a matrix is *well scaled* if the magnitudes of its nonzero elements are close to each other. Obviously, both definitions lack any quantitative measure. The tendency is that if scaling reduces the spread of the magnitudes then the problem is easier to solve. There are several counterexamples, though. One of them is a matrix with elements $a_j^i = 1 \pm \varepsilon_j^i$, where $\varepsilon_j^i > 0$ is a small random perturbation, not greater than 10^{-8}.

For measuring the spread of the magnitudes of the nonzero elements of an LP matrix, there are two methods in circulation. Fulkerson and Wolfe [Fulkerson and Wolfe, 1962] and also Orchard-Hays [Orchard-Hays, 1968] have recommended

$$\frac{\max |a_j^i|}{\min |a_j^i|}, \quad \text{over all } a_j^i \neq 0, \tag{7.38}$$

and say a matrix is well scaled if this quantity is not larger than a threshold value of about $10^6 - 10^8$. The other measure is due to Hamming [Hamming, 1971] and Curtis and Reid [Curtis and Reid, 1972]

$$\sum_{a_j^i \neq 0} (\log |a_j^i|)^2. \tag{7.39}$$

They say a matrix is well scaled if this number has an acceptable value. The smaller this value the better the scaling of the matrix is. It is not possible to give some numerical estimates for this type of acceptability because (7.39) is an absolute measure which keeps growing by the

number of nonzero entries, ν_A. A 'standard deviation like' measure can be obtained for the average magnitude if the square root of (7.39) is normalized by ν_A. The main argument in favor of using (7.39) and not (7.38) is that (7.39) is less sensitive to a single extreme value than (7.38).

While the main purpose of scaling is to increase the accuracy of processing the matrices, scaling itself can be a source of generating small errors. If the scale factors are not the powers of the base of the floating point number representation of the computer then every time a multiplication or division is performed there is a chance of introducing a round-off error. It can be avoided if the scale factors are restricted to be some powers of the base. This leaves the mantissa, and thus the accuracy, unchanged. Assuming binary representation, the nonzero coefficients of the matrix can be written in the normalized floating point form of

$$a_j^i = f_j^i \, 2^{e_j^i}, \quad \text{where } \frac{1}{2} \leq f_j^i < 1. \tag{7.40}$$

The scale factors will be powers of 2,

$$R_i = 2^{\rho_i}, \quad \text{and} \quad C_j = 2^{\gamma_j}, \tag{7.41}$$

where e_j^i, ρ_i and γ_j are integers. If we scale the matrix as in (7.36) then the exponents of the scaled elements, denoted by $\bar{e}_j^i$, become

$$\bar{e}_j^i = e_j^i + \rho_i + \gamma_j. \tag{7.42}$$

It is easy to see (by using base 2 logarithm) that minimizing (7.39) is equivalent to

$$\min_{\rho_i, \gamma_j} \; g(\boldsymbol{\rho}, \boldsymbol{\gamma}) = \sum_{a_j^i \neq 0} (e_j^i + \rho_i + \gamma_j)^2. \tag{7.43}$$

This is a least squares problem that has to be solved for ρ_i, $i = 1, \ldots, m$ and γ_j, $j = 1, \ldots, n$. For the first glance it seems to be a huge task but Curtis and Reid [Curtis and Reid, 1972] have shown a very efficient way of solving it approximately. Exact solution is not really needed since the final values have to be rounded to the nearest integers anyhow to obtain round-off free scale factors.

Before turning our attention to solving (7.43) we briefly overview other, less sophisticated, scaling methods. They are all heuristics aiming at reducing the spread of the magnitudes of the nonzeros. As such, there is no performance guarantee and, indeed, in some cases they (or some of them) may lead just to the opposite.

Equilibration. Equilibration seems to be the best established standard method. Each row is scaled to make the largest absolute element

equal 1. It is achieved by multiplying the row with the reciprocal of the largest absolute element (so the row scale factors are these reciprocals). It is followed by a similar scaling for the columns (i.e., the column scale factors are the reciprocals of the absolute column maximum values). As a result, the largest absolute element in each row and each column will be 1. This method can generate small round-off errors.

Geometric mean. For each row the

$$(\max_j\{|a_j^i|\} \times \min_j\{|a_j^i|\})^{\frac{1}{2}}$$

quantity is computed and its reciprocal (or its nearest power of 2) is used to multiply the nonzeros of the row. A similar action is taken for each column.

Arithmetic mean. Each row is multiplied by the reciprocal of the arithmetic mean of the nonzero elements in the row (or the nearest power of 2), followed by a similar action for the columns.

In practice, these simple methods are used in combination, one after the other. If round-off is not an issue the last method in a combination is always equilibration. The reason for it is that it gives some sense to setting some absolute tolerances in the LP problem. As such, a typical combination is geometric mean followed by equilibration.

An interesting dynamic scaling technique has been proposed by Benichou at al., [Benichou et al., 1977]. Their method is the choice in the IBM MPSX system. First, they apply geometric scaling one to four times. The variance of the nonzero elements of the scaled matrix, defined as

$$\frac{1}{\nu_A}\left(\sum (a_j^i)^2 - \frac{\left(\sum |a_j^i|\right)^2}{\nu_A}\right),$$

where a_j^i denotes the elements of the scaled matrix and ν_A is the number of nonzeros in $\mathbf{A}$. It is evaluated after every pass. If it falls below a given value (they recommend 10) geometric scaling is stopped. This procedure is followed by equilibration. Though this scaling can introduce small rounding errors it has become quite popular because it turned out to be quite helpful in numerically difficult cases.

Now we turn to solving (7.43) and present a special conjugate gradient method recommended by Curtis and Reid [Curtis and Reid, 1972]. We can refer to their procedure as a sort of 'optimal scaling' as it aims at finding the best scaling factors using (7.43) as the objective function. Setting the partial derivatives of g with respect to ρ_i, $i = 1, \ldots, m$ and

γ_j, $j = 1, \ldots, n$ equal to zero we obtain, after rearrangement, a set of linear equations in $m + n$ variables in the following way

$$\sum_{j:a_j^i \neq 0} (\rho_i + \gamma_j) = - \sum_{j:a_j^i \neq 0} e_j^i, \quad i = 1, \ldots, m, \tag{7.44}$$

$$\sum_{i:a_j^i \neq 0} (\rho_i + \gamma_j) = - \sum_{i:a_j^i \neq 0} e_j^i, \quad j = 1, \ldots, n. \tag{7.45}$$

Let r_i and c_j denote the nonzero count of row i and column j, respectively. (This is an exceptional use of c_j as it is usually reserved to denote the objective coefficients of the LP problems.) Then, (7.44) and (7.45) can be rewritten as

$$r_i \rho_i + \sum_{j:a_j^i \neq 0} \gamma_j = - \sum_{j:a_j^i \neq 0} e_j^i, \quad i = 1, \ldots, m, \tag{7.46}$$

$$\sum_{i:a_j^i \neq 0} \rho_i + c_j \gamma_j = - \sum_{i:a_j^i \neq 0} e_j^i, \quad j = 1, \ldots, n. \tag{7.47}$$

If we define the boolean matrix $\mathbf{Z}$ to represent the nonzero pattern of $\mathbf{A}$, i.e., $z_j^i = 1$, if $a_j^i \neq 0$ and $z_j^i = 0$ otherwise, then (7.46) and (7.47) can be written as

$$\mathbf{M}\boldsymbol{\rho} + \mathbf{Z}\boldsymbol{\gamma} = \mathbf{s} \tag{7.48}$$

$$\mathbf{Z}^T\boldsymbol{\rho} + \mathbf{N}\boldsymbol{\gamma} = \mathbf{t}, \tag{7.49}$$

where $\mathbf{M} = \text{diag}\langle r_1, \ldots, r_m \rangle$, $\mathbf{N} = \text{diag}\langle c_1, \ldots, c_n \rangle$, and $s_i = -\sum_j e_j^i$ and $t_j = -\sum_i e_j^i$ are the components of $\mathbf{s}$ and $\mathbf{t}$, respectively. In matrix form,

$$\begin{bmatrix} \mathbf{M} & \mathbf{Z} \\ \mathbf{Z}^T & \mathbf{N} \end{bmatrix} \begin{bmatrix} \boldsymbol{\rho} \\ \boldsymbol{\gamma} \end{bmatrix} = \begin{bmatrix} \mathbf{s} \\ \mathbf{t} \end{bmatrix}. \tag{7.50}$$

The matrix on the left hand side of (7.50) is singular because the sum of the m equations in (7.48) is equal to the sum of the n equations in (7.49). Additionally, this matrix is symmetric, positive semidefinite, and possesses Young's 'property A'. Therefore, a special algorithm of [Curtis and Reid, 1972] can be used that reduces both the computational effort and storage requirements. It is an iterative procedure with excellent convergence properties.

In the setup phase of the algorithm an initial solution is chosen. It is composed of the trivial solution to (7.48), $\boldsymbol{\rho}_0 = \mathbf{M}^{-1}\mathbf{s}$ and $\boldsymbol{\gamma} = \mathbf{0}$. The residual with this solution is

$$\begin{bmatrix} \mathbf{0} \\ \boldsymbol{\delta}_0 \end{bmatrix}, \quad \text{where } \boldsymbol{\delta}_0 = \mathbf{t} - \mathbf{Z}^T\mathbf{M}^{-1}\mathbf{s}.$$

Later residuals can be determined recursively by setting $h_{-1} = 0$, $\boldsymbol{\delta}_{-1} = \mathbf{0}$, $q_0 = 1$, $s_0 = \boldsymbol{\delta}_0^T \mathbf{N}^{-1} \boldsymbol{\delta}_0$ and using the following equations in a bit liberal way as $\boldsymbol{\delta}_{j+1}$ will be n dimensional if j is even and m dimensional if j is odd:

$$\boldsymbol{\delta}_{j+1} = \begin{cases} -\dfrac{1}{q_j}(\mathbf{Z}\mathbf{N}^{-1}\boldsymbol{\delta}_j + h_{j-1}\boldsymbol{\delta}_{j-1}) & \text{if } j \text{ is even,} \\ \\ -\dfrac{1}{q_j}(\mathbf{Z}^T\mathbf{M}^{-1}\boldsymbol{\delta}_j + h_{j-1}\boldsymbol{\delta}_{j-1}) & \text{if } j \text{ is odd,} \end{cases} \tag{7.51}$$

furthermore,

$$s_{j+1} = \begin{cases} \boldsymbol{\delta}_{j+1}^T \mathbf{N}^{-1} \boldsymbol{\delta}_{j+1} & \text{if } j \text{ is odd,} \\ \\ \boldsymbol{\delta}_{j+1}^T \mathbf{M}^{-1} \boldsymbol{\delta}_{j+1} & \text{if } j \text{ is even,} \end{cases} \tag{7.52}$$

and

$$h_j = q_j \frac{s_{j+1}}{s_j} \tag{7.53}$$

$$q_{j+1} = 1 - h_j, \tag{7.54}$$

for $j = 0, 1, 2, \ldots$. It is obvious from (7.51) that the dimension of subsequent $\boldsymbol{\delta}$ residuals is m and n, alternating. Consequently, the residuals of (7.50) will be

$$\begin{bmatrix} \mathbf{0} \\ \boldsymbol{\delta}_j \end{bmatrix}$$

if j is even or

$$\begin{bmatrix} \boldsymbol{\delta}_j \\ \mathbf{0} \end{bmatrix}$$

if j is odd.

There is a possibility to save computational work by updating only one of the vectors $\boldsymbol{\rho}_j$ and $\boldsymbol{\gamma}_j$ at each iteration. To determine $\boldsymbol{\gamma}_{2k+2}$ the following recursion can be used

$$\boldsymbol{\gamma}_{2k+2} = \boldsymbol{\gamma}_{2k} + \frac{1}{q_{2k} q_{2k+1}} [\mathbf{N}^{-1}\boldsymbol{\delta}_{2k} + h_{2k-1} h_{2k-2} (\boldsymbol{\gamma}_{2k} - \boldsymbol{\gamma}_{2k-2})], \tag{7.55}$$

for $k = 0, 1 \ldots$, with $h_{-2} = 0$, $\boldsymbol{\gamma}_{-2} = \mathbf{0}$, and for $\boldsymbol{\rho}_{2k+3}$ we can use

$$\boldsymbol{\rho}_{2k+3} = \boldsymbol{\rho}_{2k+1} + \frac{1}{q_{2k+1} q_{2k+2}} [\mathbf{M}^{-1}\boldsymbol{\delta}_{2k} + h_{2k-1} h_{2m} (\boldsymbol{\rho}_{2k+1} - \boldsymbol{\rho}_{2k-1})], \tag{7.56}$$

for $k = 0, 1, \ldots$, with $\rho_{-1} = \rho_1 = \rho_0$. To synchronize the two updating formulas at the termination of the iterations, one or the other of the following formulas has to be used

$$\gamma_{2k+1} = \gamma_{2k} + \frac{1}{q_{2k}}[\mathbf{N}^{-1}\delta_{2k} + h_{2k-1}h_{2k-2}(\gamma_{2k} - \gamma_{2k-2})] \tag{7.57}$$

and

$$\rho_{2k+2} = \rho_{2k+1} + \frac{1}{q_{2k+1}}[\mathbf{M}^{-1}\rho_{2k+1}h_{2k-1}h_{2k}(\rho_{2k+1} - \rho_{2k-1})]. \tag{7.58}$$

As a stopping criterion

$$s_{j+1} \leq 0.01\nu_A \tag{7.59}$$

is used where ν_A is the number of nonzeros in $\mathbf{A}$.

The above procedure may look complicated. However, it can be implemented quite easily. In practice it converges in no more than $8 - 10$ iterations regardless of the size of the problem and the time taken is just a small fraction of the ensuing simplex iterations. Very often it is sufficient to make $4 - 5$ iterations. When an approximate solution to (7.43) is obtained, all ρ_i and γ_j values are rounded to the nearest integer giving $\bar{\rho}_i$ and $\bar{\gamma}_j$. The scale factors are determined from $R_i = 2^{\bar{\rho}_i}$ and $C_j = 2^{\bar{\gamma}_j}$.

It is important to note that scaling can cause an adverse effect. For instance, there are models with a large number of ± 1 entries which is generally very advantageous for the solution algorithm. It is a source of spontaneous cancellations during factorization and updating which is a welcome event. Such a problem certainly does not benefit from scaling since many of the ± 1 values may undergo scaling and become distinct. Therefore, in an advanced implementation there must be provision for, maybe different versions of, scaling and also the possibility of solving the problem unscaled.

Scaling does not require sophisticated data structures but the row- and columnwise access to $\mathbf{A}$ is important. It can be achieved by storing $\mathbf{A}$ in both ways. This storage is very useful in several other algorithmic units, therefore, it is highly recommended if memory allows.

7.3. Postsolve

As a result of preprocessing (including problem reformulation discussed in Chapter 1) the problem actually solved is equivalent to the original one but not identical with it. Therefore, the solution obtained has to be transformed back to present it in terms of the original variables and constraints. This activity is called postsolve. During postsolve all the changes have to be undone using the values of the variables provided by the solver.

The basic requirement of postsolve is to have all changes of preprocessing properly recorded. These changes are best kept in a stack type data structure where always the most recently made action is on the top. The rationale behind it is that the changes have to be undone in the reverse order. If we perform the relevant 'undo' operation with the information on the top of the stack, it can be deleted and the next becomes the top as long as the stack is not empty.

The usual order of the actions during preprocessing is

(i) problem reformulation, see Chapter 1,

(ii) presolve, section 7.1

(iii) scaling, section 7.2.

As they involve different types of operations it is practical to set up three separate stacks. They can be implemented in many different ways. While it is not a difficult exercise, it would be difficult to say which is the best solution. We leave this issue open for the reader.

7.3.1 Unscaling

The first thing to do with the solution is to express it in terms of the unscaled problem (assuming the problem was scaled). It requires undoing the effects of scaling. Whatever combination of scaling procedures was used, ultimately it was expressed as a pre- and postmultiplication of $\mathbf{A}$ by diagonal matrices as shown in (7.36):

$$\mathbf{RAC}\bar{\mathbf{x}} = \mathbf{Rb},$$

where $\mathbf{R} = \text{diag}\langle R_1, \ldots, R_m \rangle$, $\mathbf{C} = \text{diag}\langle C_1, \ldots, C_n \rangle$, and $\mathbf{C}\bar{\mathbf{x}} = \mathbf{x}$, with $\bar{\mathbf{x}}$ representing the solution of the scaled problem. Therefore, the unscaled values of the structural variables are obtained if their computed values are multiplied by the corresponding column scale factors.

The values of the logical variables are determined by the solver to equate the left hand side with the scaled right-hand side. To restore their unscaled value they have to be divided by the corresponding row scale factors.

If the objective row was scaled the reduced costs (dual logicals) also have to be unscaled by dividing them by the column scale factors.

Finally, the shadow prices (the reduced costs of the logical variables which are the dual variables) have to be multiplied by the scale factors of the rows they belong.

7.3.2 Undo presolve

Perhaps this is the most complicated part of reversing the actions of preprocessing. Even this can be done relatively easily if the events of presolve have been properly recorded. Gondzio in [Gondzio, 1997] calls it *presolve history list* and represents every modification to the variables (both primal and dual) by a triple consisting of two integers and a double precision real number (i_1, i_2, a). The integers identify the variable(s) involved and the type of the action while the real number represents the numerical information, like value of fixing or change of the bound. For example, a lower bound correction can be recorded as $(i_1 = j,\ i_2 = 0,\ a = (\Delta)\ell_j)$. In this case the first integer stores the index of the variable, the second integer denotes the type of the operation. The real number can represent the change in the bound or the new bound itself. The latter is somewhat more practical. As another example, if an implied free variable is identified it can be stored as $(i_1 = j,\ i_2 = -i,\ a = y_i)$, where y_i comes from equation (7.25). Now the sign of the second integer defines the type of the action.

Note, fixing a variable at a certain value entails a translation of the right-hand side, see equations (1.13), (7.11) and (7.12).

Most of the steps of presolve are trivial to undo as the inverse operations are uniquely defined. This applies equally to modifications of the primal and dual variables. The important thing is to follow the strict reverse order of actions.

There is one case that deserves special attention. Namely, if the reduction of sparsity is included in presolve (section 7.1.11) then regaining the dual information requires a little care. In one step of the reduction two rows are involved in a Gaussian elimination type operation. It can also be represented by a triple $(i_1 = i,\ i_2 = k,\ a = \gamma)$, where the symbols refer to equation (7.35). If several such steps have been performed their product can be represented by a nonsingular matrix $\mathbf{M}$ (which need not be formed explicitly). As a result of this operation we have $\bar{\mathbf{A}} = \mathbf{MA}$. Therefore, the dual variables at an optimal solution satisfy

$$\mathbf{A}^T\mathbf{M}^T\bar{\mathbf{y}}^* + \mathbf{d}^* - \mathbf{w}^* = \bar{\mathbf{c}},$$

where superscript * refers to the optimal values. From this expression the dual solution of the original problem can be obtained as

$$\mathbf{y}^* = \mathbf{M}^T\bar{\mathbf{y}}^*.$$

It is our belief that the presolve techniques discussed in section 7.1 are sufficient for most cases when the problem is being prepared for the simplex method. Readers, interested in more details about presolve and

postsolve are advised to study [Andersen and Andersen, 1995], [Gondzio, 1997] where some more tests are described together with their reverse operations.

7.3.3 Undo reformulation

Development of computational forms # and #2 required very simple operations, see Chapter 1. They included multiplying a row or a column by -1, moving some finite bounds of variables to zero and creating range type constraints. They entailed possible transformations in the individual bounds of the variables, the objective coefficients and also the right-hand side elements. It was emphasized that these changes had to be recorded to enable the restoration of the original problem after solution.

If the operations are recorded on a stack they can be rewound in a reverse order. Their simplicity does not make it necessary to give an itemized description of them. Readers are referred to Chapter 1 from where all necessary steps can easily be reconstructed and implemented.